
Design Justification Report

Multi-Head Self-Attention Hardware Accelerator for the Conformer
Architecture

1. Problem and Motivation

The target kernel is **Multi-Head Self-Attention (MHSA)** from the sooftware/conformer PyTorch implementation. Profiling used torch.profiler on Intel Core Ultra 7 256V (2.2 GHz, 8 cores, Windows 11): T=1000, batch=2, d_model=64, 4 blocks, 10 runs (mean = 26.95 ms, σ = 3.58 ms).

All data in this section is from codefest/cf02/project_profile.txt. Two bottlenecks emerged. **Operator level (Section 1):** aten::mkldnn_convolution consumed 21.24% of self CPU time (52.991 ms, 140 calls = 14/pass, 137.58 MB). **Module level (Section 5):** SelfAttention averaged 58.74% of per-block time — 2.362 + 2.249 + 2.284 + 2.138 = **9.033 ms total = 33.51% of wall-clock**. Its constituent operators: aten::bmm 8.78%/21.9 ms/**458.36 MB**; aten::addmm 6.84%/17.1 ms; aten::_softmax 5.41%/13.5 ms/151.37 MB.

MHSA was chosen for hardware acceleration for three reasons: (1) its three operators combined (~21.0% self CPU) match convolution's cost, but convolution is already well-served by oneDNN/MKL-DNN; (2) Stages 2 (QK^T) and 4 (Score $\times$ V) scale as $O(T^2)$ — aten::bmm is the largest single memory consumer at 458.36 MB; (3) the 8 heads are fully independent, mapping directly onto 8 parallel chiplets with no synchronisation overhead.

2. Roofline Analysis

CPU ridge point (codefest/cf02/roofline_project.pdf): $4 \text{ P-cores} \times 4.8 \text{ GHz} \times 16 \text{ FLOPs/cycle} + 4 \text{ E-cores} \times 3.7 \text{ GHz} \times 16 = 307.2 + 236.8 = 544 \text{ GFLOPs/s}$. Memory BW (LPDDR5X spec) = **137 GB/s**. Ridge point = $544/137 = 3.97 \text{ FLOPs/byte}$.

Refer to Fig 1 - Roofline plot from /project/m4/report/figures/ fig1.png.

The ai_calculation.md FLOP formulas contain a factor-of-1000 arithmetic error in Stages 2 and 4 ($2 \times 16 \times 1000 \times 8 \times 1000 = 256,000,000$, not $256,000,000,000$ as written). Corrected total at T=1000: **609.5 MFLOPs**. The profiler (Section 3 shapes: [2,8,249,8]) shows T=249 post-subsampling inside the attention layers; corrected FLOPs at T=249 = **50.05 MFLOPs**. Both give AI below the ridge point — MHSA is **memory-bound** on the CPU (AI = 2.87 FLOPs/byte at T=249; AI = 2.32 at T=1000, both < 3.97). Measured throughput: $50.05 \text{ MFLOPs} / 9.033 \text{ ms} = 5.54 \text{ GFLOPs/s} = 1.0\% \text{ of CPU peak}$.

Scenario	AI (FLOPs/B)	Ridge	Bound	Performance
CPU — T=249 (profiler actual)	2.87	3.97	Memory-bound	5.54 GFLOPs/s measured
CPU — T=1000 (corrected)	2.32	3.97	Memory-bound	—
Accel PE level (OS, TILE=32)	16	29.2	Mem-bound at TILE=32	2,103 GFLOPs/s (sim)
Accel target (TILE=64, projected)	32	29.2	Compute-bound	~4,000 GFLOPs/s

CPU is memory-bound because PyTorch materialises Q/K/V/scores/context to DRAM between non-fused kernel launches. Accelerator eliminates this via output-stationary PEs (C stays in accumulators) and weight-stationary Chiplet 0 (W_Q/K/V/O loaded once). Per-tile AI at TILE=64 = $2 \times 64^3 / (2 \times 64^2 \times 2) = 32 \text{ FLOPs/byte} > \text{accelerator ridge } 29.2 \rightarrow \text{compute-bound}$.

3. Precision and Data Format

All inter-chiplet transfers and weight loads use **BF16** (1 sign, 8 exponent, 7 mantissa bits). PE accumulators, softmax intermediates, and Newton-Raphson reciprocal use **FP32**. FP16 was rejected — its 5-bit exponent overflows on pre-softmax attention logits. BF16 preserves FP32's dynamic range while halving UCle transfer size (8,192 vs 16,384 bytes per TILE=64 tile) and reducing multiplier area $\sim 4\times$ in GPDK045.

Quantisation error budget (project/m2/precision.md): systolic MACs: $2-3 \times 2^{-7} \approx 1.5-2.5\%$ (FP32 accumulators prevent error growth with K); softmax bf16_exp LUT: $\leq 2^{-7} = 0.78\%$ (128-entry table, single lookup — more accurate than a 3-stage Horner polynomial); NR reciprocal: $\leq 2^{-7}$ after 2 iterations (bounded by BF16 floor of sum_bf16); 5 UCle die crossings: $\leq 5 \times 2^{-7} \approx 3.9\%$ worst-case (RMS $\approx 1.7\%$). M4 simulation verified **0 mismatches** across 1,024 BF16 elements within 2 ULP tolerance.

4. Dataflow and Architecture

Two dataflow patterns operate at different granularities. At the **PE array level**, the design is **output-stationary**: FP32 partial-sum accumulators `acc_fp32[i][j]` stay in each PE across all $K=64$ steps while A columns and B rows stream in. AI per tile = $2 \times \text{TILE}^3 / (2 \times \text{TILE}^2 \times 2) = \text{TILE}/2 = 32$ at $\text{TILE}=64$. At the **system level** (Chiplet 0), the design is **weight-stationary**: $W_Q/K/V/O$ are loaded once from SRAM and reused across all T token tiles, raising system-level AI to ~ 120 FLOPs/byte. Head chiplets are tile-stationary (both operand tiles held on-chip per tile computation).

Refer to Fig 2 - Data flow diagram from `/project/m4/report/figures/ fig2.png` and Fig 4 - Block diagram from the same folder.

The 9-chiplet topology maps the five MHSA stages directly onto specialised chiplets. Chiplet 0 (`chiplet_0_qkv_outproj.sv`) runs 3 parallel systolic arrays for QKV in Stage 1 and reuses `sa_q` for OutProj in Stage 5. Chiplets 1–8 (`chiplet_head.sv`) each handle one head's QK^T (Stage 2) and $\text{Prob} \times V$ (Stage 4). Chiplet 9 (`chiplet_9_softmax.sv`) runs 8 parallel softmax pipelines (Stage 3), reducing latency from 672 cycles sequential to 84 cycles. Each systolic array has $\text{TILE} \times \text{TILE}$ PEs; at $\text{TILE}=64$ that is 4,096 PEs per array, giving $11 \text{ arrays} \times 4,096 \times 1 \text{ GHz} = \mathbf{45.06 \text{ TMAC/s peak}}$. Wave-front latency per tile: $K + 2(\text{TILE}-1) + 1 = 191$ cycles at $\text{TILE}=64$.

Memory level	Contents	Size	Latency
L1 — PE accumulators	FP32 partial sums ($11 \text{ arrays} \times \text{TILE}^2$)	22 KB @ $\text{TILE}=64$	0 cycles
L2 — On-chip registers	Chiplet 0 weights (32 KB) + head tiles (16 KB/head)	~ 212 KB total	0–1 cycles
L3 — UCle inter-chiplet	BF16 activation tiles (A+B streams)	142 flits/tile @ 2 GHz	71 ns/tile
L4 — LPDDR5X	Weights (loaded once) + input/output tokens	137 GB/s	Weight load only

5. Hardware Interface

Three interfaces are implemented. **AXI4-Lite 32-bit @250 MHz (control plane)**: 18 CSR registers (CTRL, STATUS, SEQ_LEN, D_MODEL, NUM_HEADS, WEIGHT_ADDR 64-bit, SCALE_BF16, interrupt W1C, PERF_CYCLE 64-bit, WDT). Writes complete in 4–6 clk_axi cycles. The host exposes native AXI4-Lite master ports — no bridge required. **AXI4-Stream 512-bit @250 MHz (data plane)**: 256-beat deep FIFOs (8 complete tiles buffered at TILE=32). Effective BW = 64 bytes × 250 MHz = **16.0 GB/s**; required BW (partition_rationale.md) = 8.19 GB/s → 1.95× headroom. **UCle 1.0 @2 GHz (inter-chiplet)**: 512-bit bump bus, 36 flits/tile, 18 ns/tile (at TILE=32), peak 128 GB/s per direction.

The design is not interface-bound. Interface contributions to pipeline latency: AXI-Stream in+out = 256 ns (0.4%); UCle transfers = 54 ns (0.1%); combined = **0.5% of pipeline**. Dominant bottleneck: weight load FSM ($4 \times \text{TILE}^2$ SRAM reads). UCle tile transfer (18 ns) is 4.3× faster than systolic compute per tile (95 ns at 1 GHz).

6. Verification

A layered testbench strategy covered three milestone stages. The shared oracle — `reference_model.py` — replicates `chiplet_9_softmax.sv` numerics in pure Python: LUT-based `bf16_exp` (LOG2E_Q14 fixed-point multiply,

Stage	DUT boundary	Tests	Key coverage			
M2 <code>tb_interface.py</code>	<code>axi_if (cocotb)</code>	15 tests — PASS	All 18 CSRs; AXI4-L handshake (AW+W+B ordering); IRQ W1C; 128-beat AXI-S tile assembly; perf counter			
M2 <code>tb_compute_core.py</code>	<code>compute_core (cocotb)</code>	12 tests — PASS	Reset quiescence; CSR programmability; <code>mem_req</code> FSM; IRQ end-to-end; CDC stability; <code>test_11</code> representative tile vs reference (< 2 ULP); <code>test_12</code> all-zero tile = 1/64 exact (analytical)			
M4 <code>tb_top.sv</code>	top — full chip (Questa)	7 transactions —	PASS	AXI4-L config + readback; weight load ($4 \times \text{TILE}^2$ beats); AXI-S tile input; two-phase <code>cfg_start</code> (mode=0/1); AXI-S output on <code>m_axis_tlast</code> ; 1,024 elements vs <code>ref_data.svh</code> —	0 mismatches	, 2 ULP; <code>PERF_CY CLE</code> = 16,207

128-entry table lookup, exponent reconstruction), LCG test vectors (seed=42), BF16 RNE rounding, and Newton-Raphson reciprocal. No DUT output was used as a reference value at any stage.

7. Synthesis Results

Tool: Cadence Genus 19.12-s121_1, GPDK045 typical.lib, Jun 07 2026. Design: top_NUM_HEADS8_TILE_DIM8_D_HEAD8_D_MODEL512_FIFO_D256. Clock constraint: clk_core 1 ns = 1 GHz. TILE_DIM=8 was used (TILE=16/32 exhausted 65 GB server RAM during syn_generic). Results scale as TILE² for TILE=64 estimates.

7.1 Area — area_report.txt (project/m4/synth)

Physical area (area_report_physical.rpt, Jun 07 2026, GPDK045 typical, physical library mode). Total: **15.74 mm²** (cell 12.26 mm² + net 3.48 mm²; 3,432,394 cells; avg 3.571 $\mu\text{m}^2/\text{cell}$). The **8 head chiplets dominate at 9.28 mm² (59.0%)**, averaging 1.16 mm² each. The AXI interface is the second largest at 2.80 mm² (17.8%) — almost entirely the axis_output_fifo alone (2.79 mm², 17.7%), a 256-deep $\times$ 512-bit register array. Chiplet 0 = 2.01 mm² (12.8%); chiplet 9 softmax = 1.31 mm² (8.3%). Each fp32_add_pip accumulator instance $\approx$ 7,936 μm^2 . At design-target TILE=64 ($\times$ 64 cells): estimated total $\approx$ **1,007 mm²** (linear TILE² scaling).

Sub-block	Cells	Cell Area (μm^2)	Total Area (μm^2)	% Total
8 head chiplets	2,287,318	7,062,248	9,282,499	59.0%
u_axi_if (AXI-L + FIFO)	384,176	2,339,577	2,799,387	17.8%
axis_output_fifo (inside)	382,899	2,330,887	2,789,260	17.7%
u_c0 chiplet_0 (QKV+OutProj)	424,207	1,577,215	2,014,242	12.8%
u_taylor chiplet_9_softmax	262,842	1,049,219	1,305,565	8.3%
TOTAL	3,432,394	12,258,611	15,737,880 (15.74 mm²)	100%

Dominant area contributor is the 8 systolic-array head chiplets. The axis_output_fifo unexpectedly accounts for 17.7% of total — reducing FIFO depth from 256 to 8 beats (double-buffering only) would recover $\sim$ 17% area. TILE=64 estimate is linear scaling only.

7.2 Timing — timing_report.txt (project/m4/synth)

WNS = **-701 ps** on all 10 reported paths (group: clk_core). All paths are structurally identical — one per head chiplet's systolic array — confirming the violation is symmetric. Achieved fmax = $1/(1,659 \text{ ps data path} + 43 \text{ ps setup}) \approx$ **580 MHz**. Critical path: b_stag_r_reg (fanout 194,563, the high-fanout stagger input net) $\rightarrow$ fp32_mul partial product $\rightarrow$ fp32_add carry chain $\rightarrow$ accumulator register msml_e_r_reg.

Iteration	WNS	fmax	Δ	Change
Initial	-2,171 ps	315 MHz	—	Baseline RTL
Iteration 1	-1,247 ps	445 MHz	+924 ps / +130 MHz	fp32_add pipelined 1 $\rightarrow$ 2 stages
Iteration 2 (current)	-701 ps	580 MHz	+546 ps / +135 MHz	4-bank stagger reg + split fp32_add Stage 1
Target	0 ps	1,000 MHz	+701 ps remaining	Logarithmic barrel shifter (future work)

Remaining violation is in the 27-bit variable barrel shifter in the fp32_add alignment stage (~570 ps). Closing 1 GHz requires replacing the ripple barrel shifter with a logarithmic shifter implementation.

7.3 Power — power_report.txt (project/m4/synth)

Total power (TILE=8, 1 GHz constraint, GPDK045 typical): **13.396 W**. Logic = 7.924 W (59.15%); Register = 5.472 W (40.85%); Leakage = 0.856 mW (0.01%). Register internal power (5.157 W) is the single largest sub-category — driven by 3.4M flip-flops including the high-fanout stagger registers toggling at 1 GHz. Adjusted estimate at achieved fmax 580 MHz: dynamic scales as $f^2 \rightarrow \sim 8.8 \text{ W}$ for TILE=8. Physical area and scaled power estimates for TILE=64 require re-synthesis with a physical library.

8. Benchmark Results

Measurement method: hardware PERF_CYCLE counter (PERF_CYCLE_L/H CSR) read at pipeline end in QuestaSim 2021.3_1 simulation (sim_run.log, Jun 06 2026). Parameters: TILE_DIM=32, D_HEAD=32, D_MODEL=512, h=8, T=1024, B=2, one tile exercised. All numbers are simulation-derived; no FPGA or post-synthesis measurements are available.

Metric	M1 CPU Baseline	M4 Accelerator (TILE=32, sim)	Source
Pipeline latency	9.033 ms (SA, 4 blocks)	64.8 μ s (1 tile)	profiler Sec 5 / PERF_CYCLE
Per-token latency	18.14 μ s/token (B=2, T=249)	2.026 μ s/token (B=1, TILE=32)	derived
Speedup (per-token)	1 $\times$ baseline	9.0$\times$	18.14 $\div$ 2.026
GFLOPs/s	5.54 GFLOPs/s	1,052 GFLOPs/s	68.2 MFLOPs $\div$ 64.8 μ s
Speedup (throughput)	1 $\times$ baseline	190$\times$	1,052 $\div$ 5.54
Roofline AI	2.87 FLOPs/byte	16 FLOPs/byte (OS, TILE=32)	Section 2
Roofline bound	Memory-bound (AI 2.87 < 3.97)	Mem-bound at TILE=32 (AI 16 < 29.2) Compute-bound at TILE=64 (AI 32 > 29.2)	Section 2
Energy	—	Not available (synthesis incomplete)	—

The 9.0 $\times$ per-token speedup is the primary headline figure; it is conservative and directly comparable. The 190 $\times$ GFLOPs/s ratio reflects different operating points (D_MODEL=512 vs d=64). Gap from theoretical ceiling (4,000 GFLOPs/s): measured 2,103 GFLOPs/s = 52.6% of ceiling — explained by weight load FSM overhead at TILE=32 (4,096 cycles pure overhead) and memory-bound regime (AI=16 < accelerator ridge 29.2). Both close at TILE=64.

9. What Did Not Work

- **TILE=16/32 synthesis OOM.** Both TILE=16 and TILE=32 synthesis runs were killed during syn_generic on the 65 GB server. The systolic array gate-level expansion exceeds available RAM. Forced TILE=8 for all synthesis work. Fix: hierarchical synthesis (synthesise one PE, array-replicate at P&R) or a server with ≥ 128 GB RAM.
- **Timing closure at 1 GHz not achieved.** WNS stalled at -701 ps after two iterations. The remaining bottleneck is the 27-bit ripple barrel shifter in fp32_add's alignment stage (~570 ps). Would require replacing with a logarithmic shifter — identified as future work. Functional correctness was verified at WNS = -701 ps with 0 simulation mismatches.
- **cocotb v2 API breaking changes.** M2 testbench was written for cocotb v1 (synchronous coroutines, direct signal access). PSU server had v2 installed. Fix: rewrite with async/await and int() value conversion. Lesson: pin cocotb version in requirements.txt from the start.
- **sts_done polling unreliable in tb_top.sv (M3).** sts_done is a 1-cycle pulse in clk_core (1 ns), narrower than the polling interval between AXI read transactions. The testbench consistently missed it. Fix: replaced polling with blocking on m_axis_tlast, which is sustained for the full last output beat. Lesson: use latching status bits or stream signals for completion detection, never 1-cycle pulses.
- **reference_model.py used wrong exp algorithm.** Initially implemented the standard Taylor series $p(x) = 1+x+x^2/2+x^3/6$, assuming the RTL used a polynomial. chiplet_9_softmax.sv calls bf16_exp, which uses a LUT algorithm (LOG2E_Q14 fixed-point multiply $\rightarrow$ extract k and f_bits $\rightarrow$ 128-entry table lookup $\rightarrow$ exponent field reconstruction). Mismatch caused test_11 errors of ~5–8%. Fix: read bf16_exp.sv and replicate the exact LUT algorithm in Python. Lesson: reference model must replicate the actual RTL source, not an assumed approximation.